SAVYANT

SYSTEMS

TECHNICAL ASSIGNMENT

Fresher / Trainee Software Developer

SALES ORDER & INVENTORY MANAGEMENT

INTRODUCTION

Overview & Business Scenario

A small trading company currently maintains customer records, products and orders in spreadsheets. This causes incorrect order totals, stock mismatches and difficulty tracking sales.

CORE TECHNOLOGIES

Python | JavaScript | SQL | MySQL | Git

The objective. Build a working web application for the company to manage customers, products, sales orders and stock. Users should be able to maintain master data, create orders with multiple products, confirm or cancel orders, and review sales and stock information.

SCOPE ASSUMPTIONS

- Use one currency and one stock location.
- Product quantities must be whole numbers.
- Taxes, delivery, purchasing and payment processing are outside the scope.
- Sales reports must include **Confirmed** orders only.

HOW THIS DOCUMENT IS ORGANISED

The assignment is divided into **14 parts** spanning technology setup, authentication, database design, order and stock logic, JavaScript, REST APIs, SQL reporting, CSV import, debugging, testing and Git, followed by the required **Deliverables**, the **Technical Interview Follow-Up** and **Important Instructions**. A simple, functional interface is sufficient — focus on *correct behaviour, readable code and clear error messages*.

◆ SUBMISSION DEADLINE
Within 3 calendar days of receiving the assignment

3 DAYS

SCOPE

Assignment at a Glance

01 Technology Requirements
Python backend, HTML/CSS/JavaScript frontend, MySQL database, REST APIs and Git.

02 User Login & Access
Secure login/logout with Admin and Sales User roles enforced on the backend.

03 Database Design
Design six entities with keys, constraints, indexes and decimal money fields.

04 Customer & Product Management
Maintain master data with search, filters, validation and safe deletion.

05 Sales Order Management
Create multi-item orders with backend-calculated totals and discounts.

06 Order Status & Inventory Logic
Confirm and cancel orders with transactional, concurrency-safe stock updates.

07 JavaScript Requirements
Dynamic rows, live calculations, validation and fetch-based API calls.

08 REST API
Authenticated JSON endpoints with proper status codes and clear errors.

09 Reports & SQL
Sales and stock reports plus five standalone SQL queries.

10 CSV Import
Admin-only, all-or-nothing product import with row-level error reporting.

11 Debugging Exercise
Fix a faulty total calculation, explain the cause and prove it with tests.

12 Automated Testing
Cover calculations, stock logic, permissions and CSV import with tests.

13 Git & Code Quality
Meaningful commits, a feature branch merge and clean code organisation.

14 AI-Assisted Development
Disclose AI tool usage, review approach and corrections in AI_USAGE.md.

NAVIGATE

Table of Contents

◆ **Overview & Business Scenario** 2

◆ **Assignment at a Glance** 3

Part 1 Technology Requirements 5

Part 2 User Login & Access 5

Part 3 Database Design 6

Part 4 Customer & Product Management 7

Part 5 Sales Order Management 7

Part 6 Order Status & Inventory Logic 9

Part 7 JavaScript Requirements 10

Part 8 REST API 10

Part 9 Reports & SQL 11

Part 10 CSV Import 12

Part 11 Debugging Exercise 12

Part 12 Automated Testing 13

Part 13 Git & Code Quality 14

Part 14 AI-Assisted Development 14

◆ **Deliverables** 15

◆ **Technical Interview Follow-Up** 16

◆ **Important Instructions** 17

THE ASSIGNMENT

Requirements by Part

1

PART 1

Technology Requirements

The stack your application must be built on.

AREA	REQUIREMENT
Backend	Python using Flask, Django or FastAPI
Frontend	HTML, CSS and JavaScript; frontend frameworks are optional
Database	MySQL
Version control	Git
API	REST endpoints returning JSON
Testing	Automated tests using an appropriate Python testing tool
AI tools	Allowed, with usage disclosure

A simple, functional interface is sufficient. Focus on correct behaviour, readable code and clear error messages.

2

PART 2

User Login & Access

Authenticate users and enforce role-based permissions on the server.

Implement login and logout with two roles.

ROLE	ACCESS
Admin	Manage customers and products, adjust stock, manage all orders and view reports
Sales User	View active customers and products, create and manage their own orders

PART 2 · USER LOGIN & ACCESS — CONTINUED

REQUIREMENTS

- Store passwords securely using a suitable password-hashing library.
- Protect pages and API endpoints that require authentication.
- Enforce permissions on the backend.
- Sales Users must not access another user's orders by changing a URL or order ID.
- Provide sample accounts and setup instructions without committing real credentials.

PART 3

3 Database Design

Model the core entities with keys, constraints and indexes.

Design the following entities with appropriate primary keys, foreign keys, constraints and indexes.

ENTITY	MINIMUM FIELDS
User	ID, name, unique email, password hash, role
Customer	ID, name, email, mobile, city, active/inactive status
Product	ID, unique product code, name, category, price, stock quantity, active/inactive status
Sales Order	ID, unique order number, customer, order date, status, discount percentage, subtotal, discount amount, grand total, created by
Sales Order Item	ID, sales order, product, quantity, rate, amount
Stock Movement	ID, product, movement type, quantity change, related order where applicable, reason, created by, timestamp

REQUIREMENTS

- Use **decimal types** for monetary values.
- Explain relationships and important design decisions in the README.
- Provide migrations or `schema.sql`, together with sample data.
- Prevent deletion of customers or products referenced by orders; allow deactivation instead.

PART 4

4 Customer & Product Management

Maintain master data with search, filters and safe deletion.

CUSTOMER MANAGEMENT — IMPLEMENT

- Create, view and edit customers.
- Delete customers only when no order references them.
- Search by name, email or mobile.
- Filter by active/inactive status.

PRODUCT MANAGEMENT — IMPLEMENT

- Create, view and edit products.
- Delete products only when no order or stock movement references them.
- Search by product code or name.
- Filter by category and active/inactive status.
- Validate unique product codes.
- Prevent negative prices and stock quantities.

> 🔒 **ACCESS RULE**
>
> Only **Admin** users may modify master data.

PART 5

5 Sales Order Management

Create multi-item orders with backend-calculated financial values.

Allow users to create an order by selecting an **active customer** and adding multiple **active products**.

REQUIREMENTS — THE SYSTEM MUST

- Generate a unique order number.
- Fetch the product's current price as the order item rate.
- Store that rate on the order so later product price changes do not change existing orders.
- Allow quantities to be changed and rows to be removed while the order is Draft.
- Reject duplicate products within the same order.
- Require at least one item.
- Allow a discount percentage from **0 to 20**.
- Calculate and validate all financial values on the backend.

PART 5 · SALES ORDER MANAGEMENT — CONTINUED

CALCULATION RULES

```
Item Amount      = Quantity × Rate
Subtotal         = Sum of Item Amounts
Discount Amount  = Subtotal × Discount Percentage / 100
Grand Total      = Subtotal − Discount Amount
```

Round monetary amounts to two decimal places using a consistent rule.

◆ EXAMPLE

PRODUCT	QUANTITY	RATE	AMOUNT
Keyboard	2	1,500.00	3,000.00
Mouse	3	500.00	1,500.00
Subtotal			**4,500.00**
Discount — 10%			**450.00**
Grand Total			**4,050.00**

🔒 SECURITY

Do not trust item rates, totals, status or ownership supplied by the browser.

PART 6

6 Order Status & Inventory Logic

Model the order lifecycle and keep stock consistent under concurrency.

ORDER FLOW

Draft → Confirmed

The flow must also support a **Cancelled** state, reachable from Draft or Confirmed.

STATUS TRANSITIONS

CURRENT STATUS	ALLOWED ACTION	RESULT	STOCK EFFECT
Draft	Edit	Draft	No change
Draft	Confirm	Confirmed	Deduct ordered quantities
Draft	Cancel	Cancelled	No change
Confirmed	Cancel	Cancelled	Restore deducted quantities
Cancelled	None	Cancelled	No change

REQUIREMENTS

- Draft orders must not reserve or deduct stock.
- Check stock availability when confirming an order.
- If any product has insufficient stock, reject the **entire** confirmation.
- Save the order status, stock changes and stock movement records in **one database transaction**.
- Confirmed and Cancelled orders must be read-only.
- Repeated confirmation requests must not deduct stock again.
- Repeated cancellation requests must not restore stock again.
- Handle two users attempting to confirm orders for the same limited stock without allowing negative stock.

> ✦ **THE CANDIDATE SHOULD EXPLAIN**
>
> - The approach used to prevent conflicting stock updates.

STOCK ADJUSTMENT

- Allow Admin users to increase or decrease stock with a mandatory reason.
- Every adjustment must create a stock movement record.
- Reject adjustments that would make stock negative.

PART 7

7 JavaScript Requirements

Add responsive client-side behaviour backed by server validation.

Implement the following client-side behaviours:

FEATURE	EXPECTED BEHAVIOUR
Product selection	Fetch and display the selected product's price and stock
Dynamic item rows	Add and remove order item rows without reloading the page
Live calculations	Update item amounts, subtotal, discount and grand total
Input validation	Show clear messages for missing or invalid inputs
Confirmation dialog	Ask before confirming or cancelling an order
API interaction	Use `fetch()` or equivalent and handle success and failure responses
Duplicate submission prevention	Disable the action button while a request is processing

> 🔒 **IMPORTANT**
>
> Client-side validation must be supported by backend validation.

PART 8

8 REST API

Expose secure, validated JSON endpoints.

Implement these authenticated endpoints or equivalent routes.

METHOD	ENDPOINT	PURPOSE
GET	`/api/products`	List active products with search and pagination
POST	`/api/orders`	Create a Draft order
GET	`/api/orders/{id}`	Retrieve an accessible order and its items
POST	`/api/orders/{id}/confirm`	Confirm an order and deduct stock
POST	`/api/orders/{id}/cancel`	Cancel an order and restore stock where required

PART 8 · REST API — CONTINUED

🔒 **API REQUIREMENTS**

- Apply the same validations and permissions used by the web interface.
- Return appropriate HTTP status codes.
- Return clear JSON error messages.
- Handle missing records, invalid inputs, unauthenticated requests and permission failures.
- Do not expose passwords, secrets or internal tracebacks.

Provide example requests and responses in the README or an API collection.

PART 9

9 Reports & SQL

Build accurate sales and stock reports and write standalone SQL.

Build these reports.

REPORT	REQUIRED INFORMATION
Customer Sales Summary	Customer, confirmed order count, total sales after discount
Product Sales Summary	Product, quantity sold, gross item value before order discount
Low Stock Report	Products with stock below a user-entered threshold
Stock Movement History	Product, movement type, quantity change, order, reason and timestamp

THE CUSTOMER SALES SUMMARY MUST

- Support an order-date range.
- Include customers with no Confirmed orders in that range, showing zero totals.
- Avoid counting one order multiple times because it has multiple item rows.

ALSO SUBMIT SQL QUERIES FOR

- Top three customers by Confirmed order grand total.
- Top three products by quantity sold in Confirmed orders.
- Customers who have never placed any order.
- Month-wise Confirmed sales totals, grouped by year and month.
- Products below a supplied stock threshold.

PART 10

10 CSV Import

Import products safely with all-or-nothing validation.

Implement an **Admin-only** product import using this format:

```
product_code,product_name,category,price,stock_quantity,status
PRD-001,Keyboard,Accessories,1500,20,Active
PRD-002,Mouse,Accessories,500,50,Active
```

REQUIREMENTS

- Validate required columns and each row.
- Reject duplicate product codes within the file or already present in the database.
- Reject invalid numeric values, negative values and invalid statuses.
- Display row numbers and reasons for errors.
- If any row is invalid, **import nothing**.
- Record imported opening stock as stock movements.
- Provide one valid sample CSV and one invalid sample CSV.

PART 11

11 Debugging Exercise

Find the bug, fix it and prove the fix with tests.

The following function should calculate the total amount of all items.

```
def calculate_total(items):
    total = 0
    for item in items:
        amount = item["qty"] * item["rate"]
        total = amount
    return total
```

For this input:

```
items = [
    {"qty": 2, "rate": 100},
    {"qty": 3, "rate": 50}
]
```

The expected result is **350**.

PART 11 · DEBUGGING EXERCISE — CONTINUED

✦ SUBMIT A CORRECTED FUNCTION AND A SHORT EXPLANATION COVERING

- The cause of the incorrect result.
- Behaviour for an empty list.
- Behaviour when quantity or rate is missing.
- Validation of invalid quantities and rates.
- Tests demonstrating the correction.

PART 12

12 Automated Testing

Cover the critical business logic with meaningful automated tests.

Include meaningful automated tests for at least these scenarios:

SCENARIO	EXPECTED RESULT
Order with multiple items and discount	Correct subtotal and grand total
Zero or negative quantity	Rejected
Discount outside 0–20%	Rejected
Confirmation with sufficient stock	Order confirmed and stock deducted
Confirmation with one insufficient-stock item	No stock or status changes
Repeated confirmation	No additional deduction
Cancellation of Confirmed order	Stock restored once
Sales User accesses another user's order	Access denied
CSV containing an invalid row	No products imported

Document how to run the tests. Candidates may include additional tests.

PART 13

13 Git & Code Quality

Deliver clean, well-organised source through Git.

Use Git throughout development.

INCLUDE

README.md | .gitignore | .env.example

REQUIREMENTS

- Make meaningful commits as features are completed.
- Develop at least one feature on a separate branch and merge it into the main branch.
- Include `.gitignore` and an example environment configuration file.
- Exclude real credentials, virtual environments and unnecessary generated files.
- Organize routes, business logic and database access clearly.
- Avoid duplicated calculation and validation logic.
- Use readable names and focused functions.
- Document dependencies and supported runtime versions.
- Submit the repository URL or a Git repository archive retaining its history.

PART 14

14 AI-Assisted Development

AI tools are permitted — disclose how you used them.

Submit `AI_USAGE.md` containing:

ITEM	REQUIRED DETAILS
Tools used	AI tools used, or “None”
Usage	Tasks where AI provided assistance
Example prompts	Up to three representative prompts
Review	How generated code was checked
Corrections	An issue or limitation identified and corrected, if applicable

✦ PLEASE NOTE

Candidates must be able to **explain, debug and modify** their submitted code during the technical interview.

WHAT TO SUBMIT

Deliverables

1 **Source Code**
Complete Python and JavaScript application with Git history.

2 **Database Setup**
Migrations or schema, plus sample data.

3 **README**
Setup instructions, sample account creation, assumptions and known limitations.

4 **API Documentation**
Example requests and responses or an API collection.

5 **SQL Queries**
All five requested queries.

6 **CSV Samples**
Valid and invalid import files.

7 **Automated Tests**
Test code and execution instructions.

8 **Debugging Exercise**
Corrected function, explanation and tests.

9 **AI Usage Disclosure**
`AI_USAGE.md`

10 **Demo Video**
A 5–8 minute recording showing the working application.

WHAT HAPPENS NEXT

Technical Interview Follow-Up

Shortlisted candidates will demonstrate their application, explain design decisions and complete a small live change.

✦ THE DISCUSSION WILL FOCUS ON

- Understanding of the submitted Python, JavaScript and SQL.
- Validation and error handling.
- Stock consistency and database transactions.
- Debugging approach.
- Git history and development workflow.
- Ability to adapt the application when requirements change.

READ CAREFULLY

Important Instructions

Ground rules for this assignment

- ✓ Use **Python** (Flask, Django or FastAPI) with a **MySQL** database.
- ✓ A **simple, functional interface** is sufficient — focus on correct behaviour, readable code and clear error messages.
- ✓ Calculate and validate all financial values on the **backend**; do not trust values supplied by the browser.
- ✓ Do **not commit real credentials**, virtual environments or unnecessary generated files.
- ✓ The application **must run locally** by following the README. Public hosting is optional.
- ✓ If a requirement is incomplete, **identify it clearly** and explain what remains.
- ✓ AI tools are permitted with disclosure, but you **must be able to explain, debug and modify** your submitted code during the technical interview.
- ✓ Submission naming format: **FullName_TechnicalAssignment**



Thank you for taking the time to complete this assignment.
We look forward to reviewing your solution.

SAVYANT SYSTEMS · 2026